LAB 17

Event-Driven Automation: TradingView Webhook to Signal, Risk and Execution

Item	Detail
Topic	Topic 4: Building an End-to-End Algorithmic Trading Pipeline: Backtest, Forward Test and Performance Analysis
Pipeline stage	Event-driven automation
TSC reference	A7, K5, K7
Duration	20 min
Folder	labs/Lab17-tradingview-webhook/
Files	Lab17-tradingview-webhook/webhook_stage.py, Lab17-tradingview-webhook/send_test_alert.py, Lab17-tradingview-webhook/lcc_webhook_strategy.pine, Lab17-tradingview-webhook/tradingview_alert_message.json, pipeline/webhook.py, Lab17-tradingview-webhook/solution/webhook.py
Mode	Runs ONLINE with your Alpaca paper keys in labs/.env, or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Some Lion City Capital traders design signals in TradingView. The desk wants their alerts to place paper orders automatically - but only through the pipeline: a gateway that checks a shared secret, the allow-list, a maximum size and duplicate alerts, asks the pipeline's own signal, runs the risk checks and then hands the order to the order manager, logging every request.

Objective

Build pipeline/webhook.py: a webhook receiver that turns TradingView alert JSON into a pipeline decision (validation -> signal -> risk -> execution), test it with a local alert simulator and a self-test, and configure a real TradingView strategy alert.

Before You Start

- Open a terminal in the labs folder and activate your virtual environment.
- Have your AI coding assistant open (for example Claude Code) in the labs folder if you want to rebuild the code from the prompts; the finished reference code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step-by-Step Instructions

Step 1: Set the shared secret (A7)

In labs/.env set WEBHOOK_SECRET to a long random string (for example the output of python3 -c "import secrets; print(secrets.token_urlsafe(24))"). Without it the receiver uses a demo secret and prints a warning.

Check: python -m pipeline check shows WEBHOOK_SECRET 'found (hidden)'.

Step 2: Build alert validation (A7, K5)

Ask your assistant for the Gateway class in pipeline/webhook.py.

AI prompt - paste into your AI coding assistant
Create labs/pipeline/webhook.py with class Gateway(cfg, session=None): secret from pipeline.config.secret(cfg, 'webhook_secret') (demo value 'lcc-demo-secret' if unset), broker = common.broker.get_broker(state_file='webhook_sim.json', reset=True), an OrderManager, a RiskManager and a set of processed alert ids. handle(payload) -> (http_code, body): 401 if hmac.compare_digest of the secret fails; ticker 'NASDAQ:AAPL' -> 'AAPL'; 422 if the ticker is not in risk.allowed_symbols, the action is not buy/sell, qty is not a number or not 0 < qty <= risk.max_order_qty; alert id = payload 'id' or a hash of ticker, action, qty and time; a repeated id returns 200 'duplicate ignored' (TradingView retries are safe). Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: Gateway(load_config()).handle({'secret': 'guess'}) returns (401, ...).

Step 3: Route valid alerts through signal, risk and execution (A7, K7)

Ask your assistant for the second half of handle().

AI prompt - paste into your AI coding assistant
In Gateway.handle add: pipeline_signal(ticker) = the configured strategy's latest target from pipeline.signals for that ticker; if webhook.require_signal_agreement is true and the alert disagrees, return 409; a sell with no position returns 422 (long only); RiskManager.check (kill switch, session, daily loss) failing returns 423 'blocked by risk'; otherwise target shares = held + qty for a buy or held - qty for a sell and call OrderManager.rebalance({ticker: target}, tag='tv<alert id>', units='shares') so the order is session-aware and idempotent; return 200 with order id, status, side, qty, fill price, pipeline_signal and signal_agrees. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: A valid buy returns 200 'order submitted' with pipeline_signal and signal_agrees in the body.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 4: Build the server, the simulator and the self-test (A7)

Ask your assistant for the HTTP layer.

AI prompt - paste into your AI coding assistant
In labs/pipeline/webhook.py add a standard-library http.server receiver (no Flask): POST webhook.path parses JSON (400 'body is not valid JSON' otherwise), calls Gateway.handle, appends a row to _outputs/Lab17_webhook_log.csv (time, client, alert_id, ticker, action, qty, http, result, pipeline_signal, order_id, status, detail; header written fresh when the server starts) and replies JSON; GET /health returns status, broker mode and kill switch. serve(cfg, host, port, max_requests) handles one request at a time; offline it uses a simulated regular session. test_alerts(secret) returns seven cases with expected codes: valid BUY 5 SPY (200), valid SELL 5 AMEX:SPY (200), wrong secret (401), TSLA (422), qty 5000 (422), malformed text (400), a retry of alert 1 (200 duplicate). send_test_alerts(url, secret) posts them with urllib and prints OK or MISMATCH; selftest(cfg) runs the server on a free port in a thread, sends the alerts and prints 'RESULT: n/7'. main(argv) for `python -m pipeline webhook [--port] [--max-requests] [--selftest]` and alert_sim_main(argv) for `python -m pipeline alert-sim --url --secret`. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline webhook --help shows --host, --port, --max-requests and --selftest.

Step 5: Run the self-test (A7, K5)

Run: python -m pipeline webhook --selftest.

Check: Seven alerts: 1 BUY 5 SPY -> 200 'order submitted', filled at 676.70, pipeline_signal 1, agrees; 2 SELL 5 SPY -> 200, filled at 676.57, signal_agrees false; 3 -> 401 'bad or missing secret'; 4 -> 422 'TSLA not on the allow-list'; 5 -> 422 'qty must be between 0 and 100'; 6 -> 400 'body is not valid JSON'; 7 -> 200 'duplicate ignored'; 'RESULT: 7/7 alerts handled as expected'; _outputs/Lab17_webhook_log.csv with 7 rows.

Step 6: Run the receiver and the simulator in two terminals (A7)

Terminal 1: python -m pipeline webhook --port 8080 (same as: python3 Lab17-tradingview-webhook/webhook_stage.py --port 8080). Open http://127.0.0.1:8080/health in a browser. Terminal 2: python3 Lab17-tradingview-webhook/send_test_alert.py --url http://127.0.0.1:8080/webhook. Stop the receiver with Ctrl+C.

Check: The health page shows status ok, broker offline-sim (or alpaca-paper) and kill_switch false; the simulator prints the same seven OK lines; the receiver prints each [code] and its stop summary.

Step 7: Let the pipeline's signal veto an alert (A7, K7)

In pipeline/config.yaml set webhook.require_signal_agreement: true and rerun python -m pipeline webhook --selftest. Set it back to false.

Check: Alert 2 (SELL while the pipeline's ema_cross signal for SPY is 1) now returns 409 'pipeline signal for SPY is 1: it disagrees with the alert' - RESULT 6/7, as designed.

Step 8: Prove the kill switch reaches the webhook (A7)

Run: python -m pipeline kill --on, then python -m pipeline webhook --selftest, then python -m pipeline kill --off.

Check: Alert 1 returns 423 'blocked by risk - KILL_SWITCH present - trading halted' and alert 2 returns 422 (no position to sell); RESULT 4/7 while halted.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step 9: Optional - connect a real TradingView alert (paid plan) (A7, K5)

Add Lab17-tradingview-webhook/lcc_webhook_strategy.pine to AMEX:SPY (EMA 10/30, 5 shares per order). Expose the receiver with an HTTPS tunnel (ngrok http 8080) because TradingView posts only to ports 80/443. Create one alert on the strategy: 'Order fills only', tick Webhook URL, paste https://<your-tunnel>/webhook and use tradingview_alert_message.json (with your secret) as the message. Webhooks need a paid plan and two-factor authentication.

AI prompt - paste into your AI coding assistant
Review labs/Lab17-tradingview-webhook/tradingview_alert_message.json and lcc_webhook_strategy.pine. Explain which TradingView placeholders fill ticker, action, qty and id, why the id makes retries safe, and three ways to keep the shared secret from leaking (who can see alert messages, chart sharing, logs). Do not change the files.

Check: Each real alert appears in the receiver console and in _outputs/Lab17_webhook_log.csv; online the paper order appears in the Alpaca dashboard.

Expected Output

python -m pipeline webhook --selftest: HTTP 200 (BUY 5 SPY filled at 676.70), 200 (SELL 5 SPY), 401 'bad or missing secret', 422 'TSLA not on the allow-list', 422 'qty must be between 0 and 100', 400 'body is not valid JSON', 200 'duplicate ignored'; RESULT: 7/7; with require_signal_agreement the SELL returns 409; with the kill switch on, the BUY returns 423; audit log _outputs/Lab17_webhook_log.csv.

Debrief

A webhook lets a charting platform trigger execution on a trading platform (K5), but the gateway code (A7) is where the controls live: authentication, allow-lists, size limits, duplicate protection, the pipeline's own signal, the risk layer and an audit log. Each automated order still pays spread and slippage (K7), so alert frequency matters.

Troubleshooting

Symptom	What to do
Address already in use	Another program uses port 8080; start with --port 8081 and use the same port in --url.
All alerts return 401	The receiver and sender use different secrets; both read WEBHOOK_SECRET from labs/.env, or pass --secret to the simulator.
TradingView shows 'webhook delivery failed'	Use the HTTPS tunnel URL on port 443, keep the receiver running and check the alert message is valid JSON.
Alerts return 423	The kill switch is on or the market is closed online; check python -m pipeline kill --status.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.